Guía de estudio del proyecto

Plataforma de Gestión Académica y Analítica de Rendimiento Estudiantil · Grupo Adilman
UES-FMO · Especialización en Administración de Bases de Datos e Inteligencia de Negocios
Estado al 30 de septiembre de 2026 · Defensa: 28 y 29 de octubre

Cómo usar esta guía

El objetivo no es que memorices el proyecto, sino que puedas *reconstruir el razonamiento* de cada decisión. En la defensa nadie pregunta «¿qué tablas tiene?»; preguntan «¿por qué esto así y no de la otra forma?». Esta guía está organizada alrededor de esas preguntas.

Cada sección tiene tres partes: qué hace la pieza, por qué se decidió así, y un recuadro **Compruébalo** con el comando exacto para verlo funcionando. No saltes los recuadros: haber visto el error real de permisos con tus ojos vale más que cinco lecturas del párrafo que lo explica.

1. El mapa: cómo encajan las cuatro capas
2. El modelo de datos operacional
3. La seguridad — el corazón del proyecto
4. El grafo curricular
5. La aplicación transaccional
6. El ETL y el modelo de costos
7. Lo que falta y por qué
8. Números del proyecto, de referencia rápida
9. Autoevaluación: 24 preguntas con respuesta
10. Glosario y comandos de bolsillo

1. El mapa: cómo encajan las cuatro capas

El proyecto separa deliberadamente el sistema que *opera* del sistema que *analiza*. Esa separación no es un capricho de diseño: es un requisito explícito del curso, y un tablero que consulte directamente las tablas transaccionales es motivo declarado de rechazo.

Capa 4 — Analítica
Streamlit. Consulta el *data warehouse*, nunca el esquema operacional. KPIs de costo, correlaciones, red curricular. *(Fase 4, pendiente)*

↑

Capa 3 — Integración (ETL)
Python. Lee el OLTP con un rol de solo lectura y el grafo con el driver oficial; transforma, anonimiza y carga al modelo dimensional. *(Fase 3, en curso)*

↑

Capa 2 — Sistema transaccional
FastAPI + Jinja2 + Tailwind. Matrícula, calificaciones, asistencia, reportes. La autorización la resuelve el motor, no este código. *(Fase 2, terminada)*

↑

Capa 1 — Persistencia
PostgreSQL 17 (`academico_oltp` + `auditoria`) y Neo4j 5 (malla curricular). *(Fase 1, terminada)*

La pregunta que hay que saber responder sobre este diagrama: **¿por qué dos motores?** Porque el dominio tiene dos estructuras de naturaleza distinta. Los registros académicos —quién se inscribió, qué nota sacó, si asistió— son tabulares y exigen integridad referencial: eso es PostgreSQL. La malla curricular —qué materia exige haber aprobado cuál— es un grafo dirigido, y las preguntas que le hacemos son de recorrido transitivo, no de un salto. Meter lo segundo en tablas obligaría a recursión en la aplicación.

2. El modelo de datos operacional

Trece tablas en dos esquemas. Los nombres importan menos que los grupos y las relaciones entre ellos.

Grupo	Tablas	Qué modela
Estructura académica	`departamentos`, `carreras`, `materias`	La oferta: qué se imparte y bajo qué plan
Actores	`estudiantes`, `docentes`, `usuarios`	Quiénes participan; `usuarios` son las credenciales de la app
Tiempo y oferta	`periodos_academicos`, `secciones`	Una sección es una materia impartida por un docente en un período
Hechos académicos	`inscripciones`, `evaluaciones`, `calificaciones`, `asistencias`	Lo que ocurre durante el ciclo
Costos	`costos_uv`	Costo institucional por unidad valorativa y período
Auditoría	`auditoria.log_cambios_notas`	Rastro inmutable de cambios de nota

La cadena central que hay que tener clarísima, porque de ella cuelga todo el análisis:

```
estudiante --> inscripcion --> seccion --> materia
                    |              |
                    |              +--> docente, periodo
                    |
                    +--> calificacion --> evaluacion (con su porcentaje)
                    +--> asistencia
```

Dos reglas de negocio que viven en el motor

El proyecto aplica un criterio constante: **si una regla debe cumplirse siempre, no se codifica en la aplicación, se impone en la base de datos**. Una regla en Python se salta conectándose con `psql`; una restricción del motor no.

Regla	Cómo se impone
La ponderación de las evaluaciones de una sección no puede exceder el 100 %	Disparador `trg_validar_ponderacion_evaluacion`, `BEFORE INSERT OR UPDATE`
Todo cambio de nota queda registrado con su autor	Disparador `trg_auditoria_cambio_nota`, `AFTER UPDATE`
Un estudiante no puede inscribirse dos veces en la misma sección	`UNIQUE (estudiante_id, seccion_id)`
Una nota está entre 0 y 10	`CHECK (nota >= 0.00 AND nota <= 10.00)`

Trampa de permisos que hay que conocer

`GRANT ... ON ALL TABLES IN SCHEMA` **solo afecta a las tablas que existen en ese momento**. No es una regla permanente. Por eso cada script posterior —`05` usuarios, `08` costos— vuelve a conceder los permisos de sus propias tablas. Si alguien agrega una tabla y olvida sus `GRANT`, la aplicación falla con *permission denied* aunque el script 01 «ya haya dado permisos sobre todo».

COMPRUÉBALO

```
docker exec -e PGPASSWORD="$POSTGRES_PASSWORD" academico_postgres \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "\dt academico_oltp.*"
```

3. La seguridad — el corazón del proyecto

Si solo pudieras dominar una sección de esta guía, sería esta. Es la decisión más defendible del trabajo y la que distingue el proyecto de un CRUD con roles.

El problema que resuelve

En la mayoría de sistemas, la autorización es código: `if usuario.rol == 'docente':`. Eso tiene dos debilidades. Si el programador olvida la comprobación en una ruta, no hay red de seguridad. Y cualquiera con las credenciales de la aplicación puede conectarse por fuera y hacer lo que quiera, porque la base de datos no sabe quién es quién: para ella todo es el mismo usuario.

La solución, en tres piezas

Primera: los roles son del motor, no de la aplicación. `rol_coordinador` y `rol_docente` existen en PostgreSQL con sus `GRANT`, y **no pueden iniciar sesión**: no son personas, son perfiles de permisos.

Segunda: la aplicación se conecta siempre como `rol_app`, un usuario de servicio creado `NOINHERIT`. Esa palabra es el seguro de todo el diseño: aunque `rol_app` sea miembro de los dos roles, **no hereda sus permisos**. Tiene que pedirlos explícitamente. Si el código olvidara hacerlo, la consulta falla en vez de correr con permisos de más. Falla cerrado.

Tercera: cada transacción cambia de rol antes de tocar un dato, según quién inició sesión.

```
ROLES_VALIDOS = frozenset({"rol_coordinador", "rol_docente"})

def get_db(request):
    db = SessionLocal()
    try:
        rol = (request.session.get("usuario") or {}).get("rol_db")
        if rol is not None:
            if rol not in ROLES_VALIDOS:
                raise HTTPException(status_code=403, ...)
            db.execute(text(f"SET LOCAL ROLE {rol}"))
        yield db
        db.commit()
    except Exception:
        db.rollback(); raise
    finally:
        db.close()
```

Los dos detalles de ese código por los que te pueden preguntar

¿Por qué `SET LOCAL ROLE` y no `SET ROLE`?

Porque `LOCAL` limita el cambio a la transacción en curso: al terminar, la conexión vuelve a su estado original. Sin `LOCAL`, el rol quedaría pegado a la conexión física; y como las conexiones se reutilizan desde un *pool*, la siguiente petición —posiblemente de otro usuario— heredaría el rol del anterior. Sería una fuga de privilegios silenciosa y dificilísima de reproducir.

¿Por qué esa lista blanca, si el rol ya viene de la base de datos?

Porque `SET ROLE` **no admite parámetros vinculados**. No se puede escribir `SET ROLE ?` y pasar el valor aparte: hay que interpolar el nombre en el texto de la consulta, que es exactamente la forma de una

inyección SQL. La lista blanca cierra esa puerta: solo dos cadenas literales pueden llegar a interpolarse. Es defensa en profundidad, porque el DDL además tiene un `CHECK` sobre la columna `rol_db`; pero las dos defensas protegen contra cosas distintas y ninguna sobra.

El recorrido completo de una petición

1. El usuario envía sus credenciales. La app las verifica con *bcrypt* contra `usuarios`.
2. Se guarda en la sesión —una cookie firmada— el `rol_db` de ese usuario.
3. En cada petición posterior, `get_db` abre una transacción como `rol_app`.
4. Valida el rol contra la lista blanca y ejecuta `SET LOCAL ROLE`.
5. La consulta corre. PostgreSQL acepta o rechaza según sus `GRANT`.
6. Al cerrar la transacción, el rol se revierte solo y la conexión vuelve limpia al *pool*.

La auditoría y por qué fue difícil

Cada cambio de nota se registra en `auditoria.log_cambios_notas` mediante un disparador. El problema: un disparador se ejecuta, por omisión, con los permisos de quien provoca el cambio. Y los roles de la aplicación *no tienen* permiso de escritura sobre la bitácora — precisamente porque no deben poder falsificarla. Resultado: corregir una nota fallaba con *permission denied for schema auditoria*.

La solución tentadora es conceder ese permiso, y de hecho una versión anterior del proyecto lo hizo. Pero entonces el auditado puede escribir en su propio expediente: puede insertar entradas fabricadas. Una bitácora que el auditado puede escribir no prueba nada.

La solución correcta es `SECURITY DEFINER`: la función del disparador se ejecuta con los permisos de *su dueño*, no de quien la dispara. Así escribe el rastro siempre, y ningún rol de la aplicación puede tocarlo.

> **El detalle que casi arruina ese arreglo**
>
> Dentro de una función `SECURITY DEFINER`, `CURRENT_USER` es el **dueño de la función**, no quien hizo el cambio. Como la columna `usuario_db` tenía `DEFAULT CURRENT_USER`, la bitácora habría registrado `postgres` en todas las filas y habría perdido su único valor. Por eso el rol se escribe explícitamente leyendo el parámetro `role`, que sí conserva el `SET ROLE` que hizo la aplicación.
>
> Además, toda función `SECURITY DEFINER` debe fijar su `search_path`. Si no, quien la llama podría anteponer un esquema propio y hacer que la función resuelva un nombre hacia un objeto suyo, ejecutándolo con los permisos del dueño.

COMPRUÉBALO — ESTA ES LA DEMOSTRACIÓN ESTRELLA

```
printf "SET ROLE rol_docente;\nDELETE FROM academico_oltp.inscripciones WHERE
inscripcion_id=1;\n" \
| docker exec -i -e PGPASSWORD="$POSTGRES_PASSWORD" academico_postgres \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"

ERROR:  permission denied for table inscripciones
```

Y la de auditoría: cambia una nota como `rol_docente`, reviértela como `rol_coordinador`, y consulta la bitácora. Cada fila queda atribuida a su rol, y ninguno de los dos puede insertar ni borrar entradas.

4. El grafo curricular

Dieciséis materias como nodos `:Materia`, unidas por relaciones `:REQUIERE_APROBADA`. Es un grafo dirigido acíclico: no hay ciclos porque una materia no puede exigirse a sí misma, ni directa ni indirectamente.

Las dos preguntas que le hacemos

```
MATCH (base:Materia)<-[:REQUIERE_APROBADA*1..5]-(bloqueadas:Materia)
RETURN base.codigo, count(DISTINCT bloqueadas) AS MateriasBloqueadasEnCascada
ORDER BY MateriasBloqueadasEnCascada DESC;
```

El `*1..5` es el corazón: significa «sigue la relación entre uno y cinco saltos». Con esa sola expresión se obtiene el bloqueo en cascada. El resultado más importante del proyecto sale de aquí: **Programación I condiciona diez de las dieciséis materias del plan**.

```
MATCH p = (meta:Materia {codigo:'BIN115'})-[:REQUIERE_APROBADA*]->(raiz:Materia)
WHERE NOT (raiz)-[:REQUIERE_APROBADA]->()
RETURN [n in nodes(p) | n.codigo] AS CadenaDependencia, length(p)
ORDER BY length(p) DESC;
```

Esta devuelve la *ruta crítica*: la cadena más larga desde la materia final hasta una sin prerrequisitos. Resultado: `PRG115 → PRG215 → ADS115 → SIF115 → BIN115`.

Por qué Neo4j y no MongoDB

La respuesta débil es «porque es un grafo». La fuerte reconoce que MongoDB también puede: `$graphLookup` hace recorridos recursivos. La diferencia está en lo que devuelve.

Criterio	Neo4j	MongoDB
Recorrido transitivo	Nativo: `*1..5`	Posible con `$graphLookup`
Devuelve el **camino**	Sí: `nodes(p)`, `length(p)`	No: el conjunto alcanzado, aplanado
Ruta crítica	Una consulta	Reconstruirla en la aplicación

Y la pregunta que de verdad puede incomodar: **¿por qué no un `WITH RECURSIVE` en PostgreSQL, que ya está?** Respuesta honesta: se podría, y para el bloqueo en cascada daría el mismo resultado. Pero el curso exige incorporar un motor NoSQL y, dado ese requisito, la malla es la única estructura del dominio genuinamente no relacional; es donde un motor de grafos aporta algo real en lugar de ser decorativo. Los datos transaccionales se quedan en PostgreSQL precisamente porque ahí sí son tabulares.

5. La aplicación transaccional

Estructura

Archivo	Responsabilidad
`main.py`	Aplicación, middleware de sesión, montaje de routers y estáticos
`config.py`	Credenciales desde el `.env`, nunca en el código
`database.py`	*Engine*, sesión y el `SET LOCAL ROLE` de la sección 3
`models.py`	Mapeo de todo el esquema. **Nunca** se llama `create_all()`
`routers/`	Un archivo por módulo, para que el equipo no choque en *merges*

El error más instructivo del proyecto

Durante la Fase 2, todos los usuarios se creaban inactivos y el inicio de sesión fallaba siempre. La causa: las columnas con `DEFAULT` en el DDL no estaban declaradas con `server_default` en el modelo, así que SQLAlchemy las incluía en el `INSERT` con un **NULL explícito** — y un NULL explícito *anula* el `DEFAULT` de PostgreSQL. La lección que hay que llevarse: **el error no se encontró leyendo el código, se encontró ejecutando la aplicación.** Es el argumento de por qué el proyecto verifica todo contra el sistema corriendo.

Los reportes

Dos decisiones que conviene saber justificar:

La agregación ocurre en SQL, no en Python. Los conteos y promedios se resuelven con `GROUP BY`, `HAVING` y `COUNT(CASE WHEN ...)`. Traer 36.000 filas a memoria para contarlas en un bucle sería más lento y no escalaría; el motor está optimizado justamente para eso.

El CSV se transmite en flujo. Se usa `StreamingResponse` con un generador, de modo que el archivo nunca se arma entero en memoria. Lleva además un BOM UTF-8 al inicio para que Excel no rompa las tildes: es un detalle práctico que se nota cuando el usuario abre el archivo.

Las funciones de consulta son públicas y reutilizables — `consultar_asistencia_acumulada`, `consultar_notas_finales` — con un parámetro de límite que vale `None` en la exportación. Así la vista web y el CSV comparten exactamente la misma lógica: no pueden divergir.

> COMPRUÉBALO
>
> Entra como `docente` y abre `/matricula`: debe responder **403**. Y usa `?periodo_id=1` en los reportes, porque el período por omisión no tiene asistencias registradas y se ve vacío.

6. El ETL y el modelo de costos

La capa de conexión

`etl/conexiones.py` es el punto único de entrada a los dos motores. Dos decisiones detrás:

No duplica los modelos. Reutiliza el mapeo de `backend/app/models.py` mediante un import diferido. Dos mapeos del mismo esquema se desincronizan tarde o temprano, y entonces el análisis y la aplicación reportan cosas distintas.

Se conecta con `rol_etl`, que solo tiene `SELECT`. Que el ETL no pueda escribir en el esquema operacional no depende de la disciplina de quien escriba el pipeline: lo impiden los `GRANT` del motor. El script de verificación intenta escribir *a propósito* y considera correcto que falle.

El modelo de costos

Es lo que convierte el proyecto en inteligencia de negocios. El docente fue explícito: sin el componente de costo, el tema «es más analítica que inteligencia de negocios y no es un negocio».

El cálculo se apoya en algo que ya existía: `materias.unidades_valorativas`. Solo faltaba el precio por unidad y el período al que aplica, porque ese precio cambia con el presupuesto.

```
costo de una materia = unidades_valorativas × costo_por_uv del período
```

Medida	Cuándo aplica
`costo_matricula_materia`	Siempre; es la base
`costo_reprobacion`	Cuando la inscripción termina reprobada — la medida central
`costo_repeticion`	Cuando `numero_intento > 1`

Tres decisiones de diseño que hay que poder defender:

- **La tabla vive en el esquema operacional, no en el *warehouse*.** Es dato institucional de referencia que alguien mantiene en el tiempo, el OLTP es la fuente de la verdad de la que el ETL lee todo lo demás, y el coordinador tiene razón legítima para consultarlo.
- **No hace falta una dimensión nueva.** El costo es una medida aditiva y el precio por unidad se resuelve por la dimensión período, que ya estaba prevista.
- **La vista auxiliar no calcula si la materia está reprobada.** Eso exige la nota final ponderada, cuya lógica ya vive en la aplicación. Duplicarla en SQL abriría la puerta a que el tablero reporte reprobados distintos a los del sistema transaccional.

> **Honestidad sobre la cifra**
>
> Los USD 25 por unidad valorativa son un **supuesto paramétrico**, no un dato oficial de la UES, y así está declarado en la propia columna `fuente` de la tabla. En la memoria se declara como supuesto o se sustituye por la cifra presupuestaria real. Presentarlo como dato institucional no es defendible, y cambiarlo no obliga a rehacer nada: todo se recalcula solo.

7. Lo que falta y por qué

Pendiente	Estado y dependencia
Pipeline ETL (3.2)	Extracción, anonimización SHA-256, métricas de grafo y cálculo de costos
Modelo dimensional (3.3)	Esquema estrella con las medidas de costo; bloquea al tablero
Tablero (Fase 4)	Tres historias: KPIs de costo, diagnóstico, red curricular
Memoria (Fase 5)	Cuatro entregas entre el 5 y el 23 de octubre

Defectos conocidos, que conviene reconocer antes de que los encuentren

- Solo tres de los ocho períodos tienen asistencias registradas, así que los reportes del período más reciente salen vacíos y parecen rotos.
- Ningún período está marcado como activo; la aplicación deduce el vigente.
- En equipos cuyo contenedor de Neo4j nunca se creó por conflicto de puertos, el grafo está vacío y hay que recargarlo.

8. Números del proyecto

Dato	Valor
Registros académicos totales	~290.000
Estudiantes / docentes	1.500 / 20
Inscripciones / calificaciones / asistencias	36.704 / 131.864 / 120.000
Materias / secciones / períodos	16 / 194 / 8 (2022-I a 2025-II)
Relaciones de prerrequisito en el grafo	16
Tasa de reprobación en 2022-I	24,0 % (989 de 4.122)
Costo de la reprobación en 2022-I	USD 98.900
Materias más costosas	MAT315 (USD 12.600), BAD215 (USD 12.000)
Materias que bloquea Programación I	10 de 16
Estudiantes en riesgo por ausentismo (2022-I)	644, con umbral de 3 ausencias
Umbrales del sistema	Aprobación 6,00 · Riesgo 3 ausencias

9. Autoevaluación

Respóndelas en voz alta antes de leer la respuesta. Si tardas más de veinte segundos en una, esa es la sección que tienes que repasar.

Arquitectura

1. ¿Por qué el tablero no consulta directamente las tablas transaccionales?

> Porque separa dos cargas de trabajo distintas: la operacional exige escrituras rápidas y normalización; la analítica, lecturas agregadas sobre datos desnormalizados. Además el curso lo declara motivo de rechazo, y consultar el OLTP con cargas analíticas degrada el sistema que la institución usa para operar.

2. ¿Por qué dos motores de base de datos?

> Porque el dominio tiene dos estructuras distintas: registros tabulares con integridad referencial, y una malla de prerrequisitos que es un grafo dirigido cuyas consultas relevantes son transitivas.

3. ¿Qué es un data mart y dónde aplica aquí?

> Un subconjunto del *warehouse* orientado a un área. Aquí el corte natural es por departamento, que ya existe como tabla desde la Fase 1.

Seguridad

4. ¿Por qué `rol_app` es `NOINHERIT`?

> Para que no herede automáticamente los permisos de los roles de los que es miembro. Debe pedirlos con `SET ROLE`. Si el código olvidara hacerlo, la consulta falla en vez de correr con permisos de más: falla cerrado.

5. ¿Qué pasaría si se usara `SET ROLE` en vez de `SET LOCAL ROLE`?

> El rol quedaría pegado a la conexión física. Como las conexiones se reutilizan desde un *pool*, la siguiente petición heredaría el rol del usuario anterior.

6. ¿Por qué hace falta una lista blanca de roles si el valor viene de la base de datos?

> Porque `SET ROLE` no admite parámetros vinculados: el nombre del rol hay que interpolarlo en el texto de la consulta, que es la forma de una inyección SQL. La lista blanca garantiza que solo dos cadenas literales puedan interpolarse.

7. ¿Por qué el disparador de auditoría es `SECURITY DEFINER`?

> Para que escriba la bitácora con los permisos de su dueño. Los roles auditados no tienen escritura sobre ella, justamente para que no puedan falsificar el rastro; sin `SECURITY DEFINER`, cualquier corrección de nota fallaría.

8. ¿Por qué no basta con dar permiso de `INSERT` sobre la bitácora?

> Porque entonces el auditado podría escribir entradas fabricadas en su propio expediente. Una bitácora que el auditado puede escribir no prueba nada.

9. Dentro de una función `SECURITY DEFINER`, ¿quién es `CURRENT_USER`?

> El dueño de la función, no quien la dispara. Por eso el rol real se registra leyendo el parámetro `role`; si se hubiera confiado en el `DEFAULT CURRENT_USER` de la columna, la bitácora diría `postgres` en todas las filas.

10. ¿Por qué toda función `SECURITY DEFINER` debe fijar su `search_path`?

> Porque si no, quien la llama puede anteponer un esquema propio y hacer que la función resuelva un nombre hacia un objeto suyo, que se ejecutaría con los permisos del dueño.

11. ¿Cómo demuestras en vivo que la autorización está en el motor?

> `SET ROLE rol_docente;` seguido de un `DELETE` sobre `inscripciones`. El error lo produce PostgreSQL, no la aplicación.

Datos y modelo

12. ¿Por qué `GRANT ON ALL TABLES` no basta?

> Porque solo afecta a las tablas existentes en el momento de ejecutarlo; no cubre las que se creen después.

13. ¿Por qué nunca se llama `create_all()`?

> Porque la fuente de verdad del esquema son los scripts de `database/oltp/`. Dejar que el ORM cree tablas produciría un esquema distinto del versionado.

14. ¿Qué es `server_default` y por qué importa?

> Marca que la columna tiene `DEFAULT` en la base. Sin ella, SQLAlchemy la incluye en el `INSERT` con un NULL explícito, y un NULL explícito anula el `DEFAULT`. Provocó que todos los usuarios se crearan inactivos.

15. ¿Cómo se calcula la nota final?

> Suma de (nota × porcentaje de la evaluación) dividida entre la suma de los porcentajes evaluados. Un disparador garantiza que la ponderación de una sección no exceda el 100 %.

16. ¿Por qué la agregación se hace en SQL y no en Python?

> Porque el motor está optimizado para eso y evita traer decenas de miles de filas a memoria.

Grafo

17. ¿Qué significa `*1..5` en la consulta de Cypher?

> Que siga la relación entre uno y cinco saltos: es el recorrido transitivo que produce el bloqueo en cascada.

18. ¿Por qué Neo4j y no MongoDB?

> MongoDB puede recorrer con `$graphLookup`, pero devuelve el conjunto de documentos alcanzados, no el camino que los une. El camino es la respuesta que necesita el reporte de materias críticas.

19. ¿Por qué no un `WITH RECURSIVE` en PostgreSQL?

> Se podría, y daría el mismo resultado para el bloqueo en cascada. El curso exige un motor NoSQL y, dado ese requisito, la malla es la única estructura del dominio genuinamente no relacional.

ETL, costos y alcance

20. ¿Cómo se garantiza que el ETL no escriba en el OLTP?

> Con los `GRANT`: `rol_etl` solo tiene `SELECT`. El script de verificación intenta escribir a propósito y espera que falle.

21. ¿De dónde sale el costo por unidad valorativa?

> Es un supuesto paramétrico, declarado como tal en la columna `fuente` de la tabla. Si aparece la cifra oficial se sustituye y todo se recalcula.

22. ¿Por qué la tabla de costos está en el OLTP y no en el *warehouse*?

> Porque es dato institucional de referencia que se mantiene en el tiempo, y el OLTP es la fuente de la que el ETL lee todo lo demás.

23. ¿Por qué el proyecto no incluye analítica predictiva?

> Porque el alcance se limita a los niveles descriptivo y diagnóstico. Predecir quién desertará es analítica de datos, no inteligencia de negocios, y quedó fuera por definición del curso.

24. Si identificas estudiantes en riesgo, ¿no es eso predecir?

> No: se identifican quienes *ya presentan* el patrón en datos históricos observados —tres o más ausencias y nota bajo el mínimo—. Es responder «por qué ocurrió», no anticipar lo que ocurrirá.

10. Glosario y comandos de bolsillo

Término	En una frase
OLTP	Sistema de procesamiento transaccional: registra las operaciones del día a día
OLAP	Procesamiento analítico: consultas agregadas sobre datos históricos
ETL	Extracción, transformación y carga desde el sistema operacional al analítico
Esquema estrella	Una tabla de hechos rodeada de dimensiones desnormalizadas
Tabla de hechos	Los sucesos medibles, con sus medidas numéricas
Dimensión	El contexto por el que se filtra o agrupa un hecho
Data mart	Subconjunto del *warehouse* orientado a un área o departamento
RBAC	Control de acceso basado en roles
`NOINHERIT`	Un rol que no hereda permisos de los roles a los que pertenece
`SECURITY DEFINER`	Función que se ejecuta con los permisos de su dueño
Unidad valorativa	Unidad de medida académica; base del cálculo de costo
Materia cuello de botella	Aquella de la que dependen muchas otras en la malla

Comandos

```
docker compose up -d                      # levantar los dos motores
.venv/bin/python -m etl.verificar_conexiones   # las tres comprobaciones del ETL
cd backend && uvicorn app.main:app --reload    # la aplicación en :8000
docker exec -i ... < scripts/analisis/costo_reprobacion.sql   # las cifras de costo
```

Cómo estudiar esto en tres sesiones

Primera: secciones 1 a 3, con los recuadros ejecutados. La seguridad es lo que más peso tiene. **Segunda:** secciones 4 a 6, y corre las consultas de Cypher y el script de costos hasta entender cada línea. **Tercera:** la autoevaluación completa en voz alta, y después el guion de defensa de `docs/recorrido_del_sistema.md`.